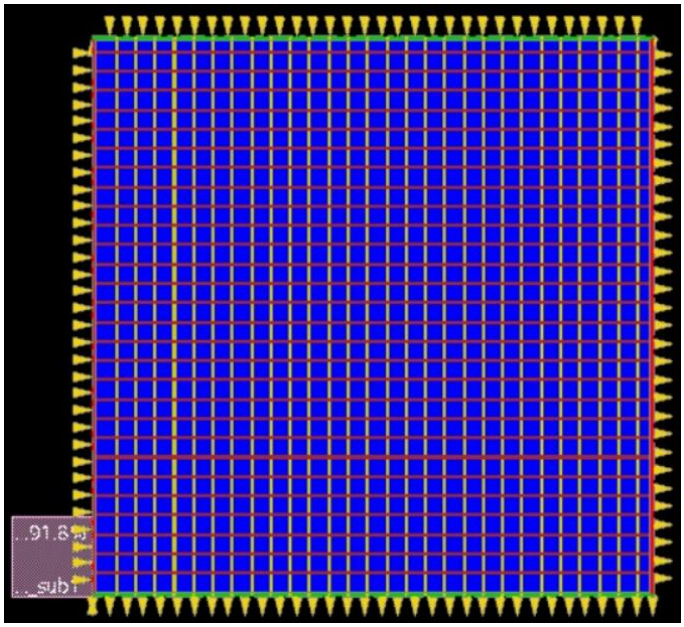
CVSD Final Project Report

R13943023 歐佾宸
R13943177 陳芷寧

1. APR Results

I. Layout:



II. DRC/LVS results

```
VERIFY DRC ..... Sub-Area: {364.480 182.240 546.720 364.480} 11 of 64  Thread : 0
VERIFY DRC ..... Sub-Area: {182.240 911.200 364.480 1093.440} 42 of 64  Thread : 7
VERIFY DRC ..... Sub-Area: {911.200 546.720 1093.440 728.960} 30 of 64  Thread : 4
VERIFY DRC ..... Sub-Area: {1093.440 182.240 1275.680 364.480} 15 of 64  Thread : 8
VERIFY DRC ..... Sub-Area: {0.000 546.720 182.240 728.960} 25 of 64  Thread : 9
VERIFY DRC ..... Sub-Area: {1275.680 911.200 1452.680 1093.440} 48 of 64  Thread : 6
VERIFY DRC ..... Sub-Area: {0.000 1275.680 182.240 1452.220} 57 of 64  Thread : 5
VERIFY DRC ..... Sub-Area: {364.480 546.720 546.720 728.960} 27 of 64  Thread : 2
VERIFY DRC ..... Sub-Area: {911.200 911.200 1093.440 1093.440} 46 of 64  Thread : 1
VERIFY DRC ..... Sub-Area: {546.720 1093.440 728.960 1275.680} 52 of 64  Thread : 3
VERIFY DRC ..... Sub-Area: {182.240 182.240 364.480 364.480} 10 of 64  Thread : 0
VERIFY DRC ..... Sub-Area: {911.200 182.240 1093.440 364.480} 14 of 64  Thread : 8
VERIFY DRC ..... Sub-Area: {1275.680 1093.440 1452.680 1275.680} 56 of 64  Thread : 6
VERIFY DRC ..... Sub-Area: {546.720 182.240 728.960 364.480} 12 of 64  Thread : 7
VERIFY DRC ..... Sub-Area: {182.240 546.720 364.480 728.960} 26 of 64  Thread : 2
VERIFY DRC ..... Sub-Area: {546.720 1275.680 728.960 1452.220} 60 of 64  Thread : 3
VERIFY DRC ..... Sub-Area: {182.240 1093.440 364.480 1275.680} 50 of 64  Thread : 5
VERIFY DRC ..... Sub-Area: {1093.440 546.720 1275.680 728.960} 31 of 64  Thread : 9
VERIFY DRC ..... Sub-Area: {728.960 546.720 911.200 728.960} 29 of 64  Thread : 4
VERIFY DRC ..... Sub-Area: {911.200 1093.440 1093.440 1275.680} 54 of 64  Thread : 1
VERIFY DRC ..... Sub-Area: {1275.680 1275.680 1452.680 1452.220} 64 of 64  Thread : 6
VERIFY DRC ..... Sub-Area: {182.240 1275.680 364.480 1452.220} 58 of 64  Thread : 5
VERIFY DRC ..... Sub-Area: {728.960 1093.440 911.200 1275.680} 53 of 64  Thread : 1
VERIFY DRC ..... Sub-Area: {1093.440 1275.680 1275.680 1452.220} 63 of 64  Thread : 6
VERIFY DRC ..... Sub-Area: {911.200 728.960 1093.440 911.200} 38 of 64  Thread : 9
VERIFY DRC ..... Sub-Area: {546.720 728.960 728.960 911.200} 36 of 64  Thread : 4
VERIFY DRC ..... Sub-Area: {364.480 1275.680 546.720 1452.220} 59 of 64  Thread : 5
VERIFY DRC ..... Sub-Area: {728.960 1275.680 911.200 1452.220} 61 of 64  Thread : 1
VERIFY DRC ..... Thread : 5 finished.
VERIFY DRC ..... Thread : 2 finished.
VERIFY DRC ..... Thread : 7 finished.
VERIFY DRC ..... Thread : 0 finished.
VERIFY DRC ..... Sub-Area: {1093.440 911.200 1275.680 1093.440} 47 of 64  Thread : 9
VERIFY DRC ..... Thread : 9 finished.
VERIFY DRC ..... Sub-Area: {911.200 1275.680 1093.440 1452.220} 62 of 64  Thread : 1
VERIFY DRC ..... Thread : 1 finished.

 Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:01:07  ELAPSED TIME: 9.00  MEM: 147.8M) ***
```

```
innovus 6> VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Mon Dec 16 18:41:46 2024

Design Name: ed25519
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (1452.6800, 1452.2200)
Error Limit = 1000; Warning Limit = 50
Check all nets
Use 10 pthreads

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Mon Dec 16 18:41:54 2024
Time Elapsed: 0:00:08.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:20.3  MEM: 0.000M)
```

```
innovus 6>
******* START VERIFY ANTENNA ********
Report File: ed25519.antenna.rpt
LEF Macro File: ed25519.antenna.lef
5000 nets processed: 0 violations
10000 nets processed: 0 violations
15000 nets processed: 0 violations
20000 nets processed: 0 violations
25000 nets processed: 0 violations
30000 nets processed: 0 violations
35000 nets processed: 0 violations
40000 nets processed: 0 violations
45000 nets processed: 0 violations
50000 nets processed: 0 violations
55000 nets processed: 0 violations
60000 nets processed: 0 violations
65000 nets processed: 0 violations
70000 nets processed: 0 violations
75000 nets processed: 0 violations
80000 nets processed: 0 violations
85000 nets processed: 0 violations
90000 nets processed: 0 violations
95000 nets processed: 0 violations
100000 nets processed: 0 violations
105000 nets processed: 0 violations
110000 nets processed: 0 violations
115000 nets processed: 0 violations
120000 nets processed: 0 violations
125000 nets processed: 0 violations
130000 nets processed: 0 violations
135000 nets processed: 0 violations
140000 nets processed: 0 violations
Verification Complete: 0 Violations
******* DONE VERIFY ANTENNA ********
(CPU Time: 0:00:19.2  MEM: 0.000M)
```

2. Algorithm Design

演算法的部分，我們是根據 final_note 簡報以及助教提供的 ed25519.py 的算法去實做，一樣是使用 Projective representation 方式進行 Point doubling 和 Point addition 的操作，以下會提出過程中實作上的演算法的細節，以及優化的方式。

I. Algorithm provided by TA:

首先簡要提點助教提供的流程，EdDSA 需要實作 M 次的點加運算如下。

$$f_M(x,y) = M * P = (x_p, y_p) + \cdots + (x_p, y_p)$$

其中 M 高達 255 位數，不可能真的去一次次相加，因此使用指數冪累加的方式，從 M 的 MSB 往 LSM 掃，每次都先 doubling，並且如果 bits 是 1 就做 addition。如此可以把運算次數的上界壓到 255 次 doubling 和 255 次 addition，大幅縮短總時間。

接著說明每個點加運算是如何實作的，由於直接運算 $(x_1, y_1) + (x_1, y_1)$ 會需要除法運算，而每次除法運算需要計算 q-2 次方，即使做指數快速冪也需要 255 次運算太花時間了。因此使用 Projective 表示方式引入第三個變數 z 來輔助計算 $(x_1 = X_1/Z_1,\ y_1 = Y_1/Z_1)$，公式如下。

$$(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2) = (X_3, Y_3, Z_3)$$
$$X_3 = Z_1 Z_2 (X_1 Y_2 + X_2 Y_1)(Z_1^2 Z_2^2 - d X_1 X_2 Y_1 Y_2)$$
$$Y_3 = Z_1 Z_2 (Y_1 Y_2 - a X_1 X_2)(Z_1^2 Z_2^2 + d X_1 X_2 Y_1 Y_2)$$
$$Z_3 = (Z_1^2 Z_2^2 - d X_1 X_2 Y_1 Y_2)(Z_1^2 Z_2^2 + d X_1 X_2 Y_1 Y_2)$$

而上面步驟會需要兩個數字的加減乘法取 mod q 的結果，加減法取 mod 很容易，但乘法則需要用到 Montgomery multiplication (MM)。公式如下

$$MM(x,y) = \frac{xy}{R} mod\ q = t, (t - q\ \ if\ \ t > q)$$

$$t = \frac{xy + (-xyq^{-1} mod\ R)q}{R}$$

$$x * y\ mod\ q = MM(MM(x,y), R^2 mod\ q)$$

如此就可以只依靠幾十個加減乘法基礎運算就實現 $(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$。最後則是把 (X, Y, Z) 映射回 (x, y)。這部分需要計算除法 X/Z 和 Y/Z，如同上面提到的，可以用$X * Z^{(q-2)}$並配合指數快速冪來達成。以上就是我們最開始嘗試實作的算法。接下來是我們優

化的方法。

II. Baseline Estimate:

我們認為乘法器的數量、大小、執行時間會是這個題目的關鍵。因此我們先估計一下過程中 MM 的次數，並以此來評估每項優化的效果。如下圖紅線所示，每次點加需要 13 次乘法運算，而需要計算 255 + M 的 1bits 次數 (用 400 來計算)，以及最後計算 X/Z 和 Y/Z 需要做 400*2 次，總共 400*14*2 (2 次MM) = 11200 次MM。

$$X_3 = Z_1Z_2(X_1Y_2 + X_2Y_1)(Z_1^2Z_2^2 - dX_1X_2Y_1Y_2)$$
$$Y_3 = Z_1Z_2(Y_1Y_2 - aX_1X_2)(Z_1^2Z_2^2 + dX_1X_2Y_1Y_2)$$
$$Z_3 = (Z_1^2Z_2^2 - dX_1X_2Y_1Y_2)(Z_1^2Z_2^2 + dX_1X_2Y_1Y_2)$$

III. Addition & Doubling:

我們發現，助教給的連結裡，Projection 表示中相乘的次數是 12 次。這主要是因為 $Y_1Y_2 + X_2Y_1$ 可以用$(X_1 + Y_1)(X_2 + Y_2) - X_1Y_2 - X_2Y_1$ 實作，如此就可以少一個乘法。

Doubliing 的部分由於我們當時認為雖然乘法減少到 8 次，但會多出很多加法運算，整體 cycle 數不見得划算。後來臨近 deadline 我們仔細分析後才發現乘法器面積的比重高達總面積的 80%，相比之下加法器的大小微乎其微，完全可以多用幾個加法器來加速，卻也沒時間修改了，這部分算是我們的遺憾。

IV. X/Z, Y/Z Division:

因為一開始是按照 ed25519.py 寫的，FSM 分兩個 state 先算 X/Z 再算 Y/Z，後來發現 Z^{-1}可以做完存著只需要算一次就好，可以節省約 400 次相乘。

V. MM (v1):

在所有MM中，$MM(R^2)$的運算就佔了一半，能不能不要算？我們發現，既然每次算完會變成$MM(x, y) = \frac{xy}{R}$，那為什麼不先乘以 R 呢？修改完後變成 $MM(xR, yR) = \frac{xRyR}{R} = xyR$ 多出了一個 R，但這其實是好事，因為我們就不用特地乘以 R 了，而是保持每個運算結果都是乘上一倍 R 的狀況。因此只要在最開頭乘上 R，在結尾用 $MM(x, 1)$ 把 R 去掉就好

了。如此一來可以省下幾乎一半的MM。需要特別注意的是常數 d 也要事先乘以 R。

VI. MM (v1):

MM本身需要三次乘法運算，每次乘法基本上至少需要一個 cycle，算上後面 mod q 也需要一個 cycle，這其實滿浪費時間的。後來我們想到，$q = 2^{255} - 19$，代表$2^{255} \bmod q \equiv 19$，這是一個很好的性質，代表長整數 X[WIDTH-1:] mod q ≡ X[254:0] + X[WIDTH-1:255] * 19，只需要一次加法運算和常數乘法就可以大幅減少 X 的長度。再做一次運算又可以再降一次。於是我們想到了不做MM(三次乘法)，改做乘法+兩次(加法+常數乘法)的方式，因為後者可以 pipeline，而前者如果要 pipeline，就需要生三個乘法器，整體 AT 不會比較好。

然而這個方式其實有個缺陷，就是我們無法保證結果會< q，因為第二次 A[WIDTH-1:255] * 19 這個運算還是可能讓結果超出 2^{255} 一點點，儘管機率很小。所以應該再後面加上判斷是否小於 q 嗎？

其實不用，說到底我們根本不需要保證每次運算結果都小於 q，只要落在可控的區間就好了。因此我們仔細分析，如果 X 和 Y 都超出 q 一點的狀況。

令 $X = 2^{255} + L_X, \quad Y = 2^{255} + L_Y, \quad L_X, L_Y << 2^{255}$

則 $XY = (2^{255} + L_X)(2^{255} + L_Y) = 2^{510} + (L_X + L_Y) * 2^{255} + L_X L_Y$

第一次運算 $XY^{(1)} = \left(2^{255} + (L_X + L_Y)\right) * 19 + L_X L_Y$

$$< 2^{255} * 32 = 2^{260}$$

第二次運算 $XY^{(2)} < 2^5 * 19 + S$，其中S為某個小於2^{255}的數。

$$< 32 * 19 + 2^{255}$$

其結果顯示如過輸入比2^{255}多一點，輸出還是會保持小於 2^{255} 加上遠小於2^{255}的數，使用 256 bits 來存是足夠了。整個運算過程中也不會有夠多次連續的加法把這個數字加到會發散的程度。

後來我們有去統計運算完超過 q 的比例，第一次運算完超過 q 的比例大約 5%，第二次運算完超過 q 的事件在數十筆測資中的所有乘法運算中一次都沒有發生過，其實滿合理的，因為成立條件是在加上一個小於$32 * 19$的數字時，要能讓 255bits 的數字進位，也就是後

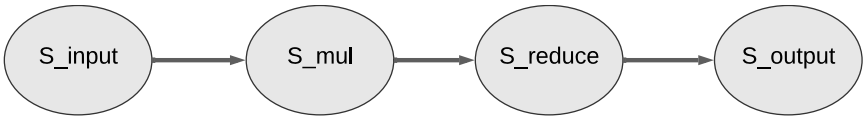
面那個數字必須滿足第 255 到第 10 個 bits 都是 1，機率小的可憐。但因為不敢保證助教會不會故意出這樣的測資，所以最後面 output 輸出前還是有檢查是否小於 q 的部分，用 2 個 cycle 求個心安。

3. Hardware Implementation

硬體實作的部分，會先以 FSM 說明整體的流程，再來則是會提到使用到的 submodule，最後則是附上點加 $(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$ 對於乘法器和加法器的調用情況。

I. FSM:

A. Top-level FSM:



上圖是整體的 FSM，共分成 4 個 states。

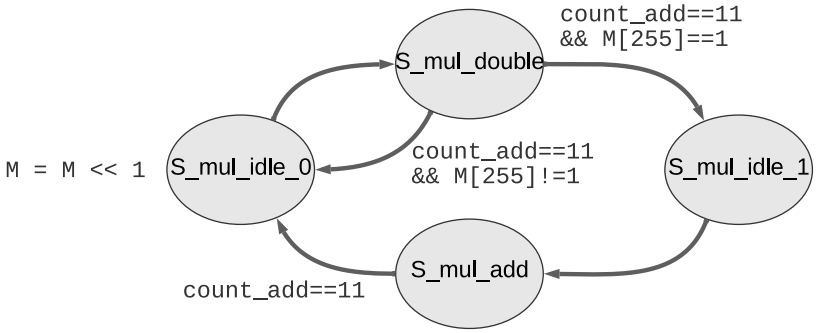
S_input:
讀取輸入並儲存。由於輸入方式是從 M->X->Y、MSB 到 LSB。因此實作時是使用 shift register 的方式，固定把資料讀取到 Y[63:0]，並且每次讀取時往左 shift 64 bits。以 verilog 語法表示就是 {M[255:0],X[255:0],Y[255:0]} <= {M[191:0], X[255:0], Y,[255:0], input[63:0]}。

S_mul:
完成 $M * (X_1, Y_1, 1)$ 的計算，實作方式是使用另一個 FSM，會在 **B.**部分說明。

S_reduce:
完成 $(X_1, Y_1, Z_1) -> (X_1, Y_1)$ 的投射。實作方式也是使用另一個 FSM，會在 **C.**部分說明。

S_output:
輸出最後答案。和輸入時同樣是使用 shift register 實作。**S_reducer** 計算完答案分別被存在 X, Y 裡面，每次固定輸出 X[255:192]，並同時往左 shift 64 bits。以 verilog 語法表示就是 {X[255:0], Y[255:0]} <= {X[191:0], Y[255:0], 64'b0};

B. Sub-FSM for S_mul:



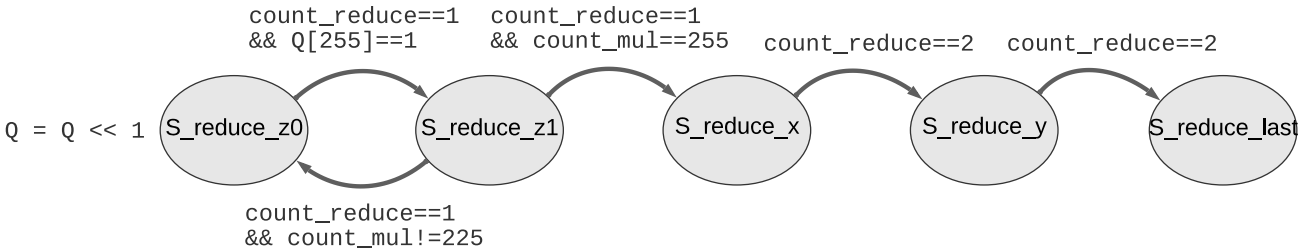
上圖是 **S_mul** 的 sub-FSM，共分成 4 個 states，皆以 S_mul 開頭以供辨認。整個流程需要做 255 次就完成 $M * (X_1, Y_1, 1)$ 的計算。

S_mul_idle_0:
將資料 (X_2, Y_2, Z_2) assign 成 (X_1, Y_1, Z_1)，為下一個 state **S_mul_double** 做準備。

S_mul_idle_1:
將資料 (X_2, Y_2, Z_2) assign 成 $(X_{input}, Y_{input}, 1)$，為下一個 state **S_mul_add** 做準備。

S_mul_add & S_mul_double:
一開始寫原本以為之後會改成兩個不同功能的 state 分別執行 addition 和 doubling，但後來衡量之下沒有做，因此目前這兩個 state 功能是一樣的。都是需要做 12 次乘法，需要 12 個 cycle，因此 **S_mul_double** 會在 count_add==11 時判斷下運算是要做 addition 還是 doubling 並跳到對應的 idle state，而 **S_mul_add** 則不需要額外判斷，因為 addition 做完一定是接 doubling。

C. Sub-FSM for S_reduce:



上圖是 **S_reduce** 的 sub-FSM，共分成 5 個 states，皆以 S_reduce 開頭以供辨認。

S_reduce_z0 & S_reduce_z1:
這兩個 state 的邏輯和 **S_mul_add & S_mul_double** 很像，都需要檢查 bit 來判斷下個 state 是要做哪個運算。**S_reduce_z0** 是把目前結果做平方，**S_reduce_z1** 則是結果乘上 z。完成 255 次後就會得到 z^{-1}。

S_reduce_x & S_reduce_y:

因為只有一個乘法器，X/Z 和 Y/Z 不能同時計算，因此分兩個 state 分別計算。

S_reduce_y & S_last:

加法和乘法則可以同時計算，因此當 **S_reduce_y** 在計算 Y/Z 時，可以處理 X/Z 的結果。而 **S_last** 則處理 Y/Z 的結果。兩者都需要做 >q 以及奇偶性的檢查並修正。

II. Basic Operation Hardware & self-defined submodules:

電路中，乘法器我們是直接用 DesignWare 的 DW02_mult 來建構一個 256bits 的乘法器。其他還有使用我們定義的 submodule “ADD_SUB” 和 “MAC”。

ADD_SUB 的功能是根據輸入的 addsub 訊號做加法或減法，並檢查結果是否超出範圍並修正。

submodule “ADD_SUB”

```
20      always_comb begin
21          if (addsub == 0) begin
22              out1 = a + b;
23          end
24          else begin
25              out1 = a - b ;
26          end
27          if (out1 >= mod) begin
28              if (addsub == 0)
29                  out1 -= mod;
30              else
31                  out1 += mod;
32          end
33      end
```

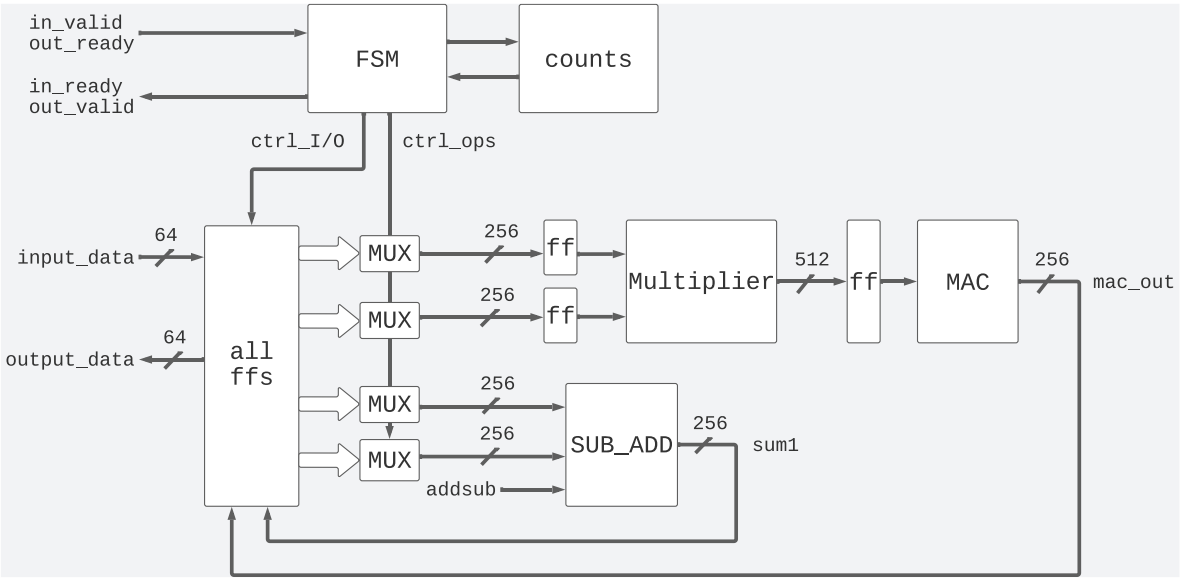
MAC _SUB 則是用來時做上面提到的把 MM 化簡需要的兩次運算。可以看到因為第一次運算的結果的上界是 2^{260}，因此 reg0 的長度要設成 256+4=260 bits。

submodule “MAC”

```
47      wire [WIDTH+4:0] reg0;
48      assign reg0 = a[254:0] + a[WIDTH*2-1:255] * 19;
49      assign out = reg0[254:0] + reg0[WIDTH+4:255] * 19;
```

III. Utilization of subblock:

A. Block diagram:



上圖是整個電路的 Block diagram，主要分成上方由 FSM 和 counters (包含 count_input, count_add, count_reduce, count_mul) 組成的 control flow 和下方的 data flow。基本模式就是根據目前的 state 和 counter 資訊來控制輸入乘法器和加減法器的 data，以及把運算結果存回正確的位置。

有幾個需要注意的點。首先 256bits 的乘法器是整個電路的 critical path，因此我們特地把乘法器的前後邏輯都清空，直接和 register 的輸入輸出對接。再來就是整個乘法包含取值、MUL、以及 MAC 總共需要三個 cycle 的時間。需要確保過程中的 dependency，因此需要妥善安排 $(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$ 的 11 個乘法以及加法計算的順序。

下面會針對我們覺得最複雜的 $(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$ 詳細展示運算流程。至於其餘部分如 reduce $(X_1, Y_1, Z_1) -> (X_1, Y_1)$ 就是單純乘法和加減法，

B. $(\mathbf{X_1, Y_1, Z_1}) + (\mathbf{X_2, Y_2, Z_2})$ Data flow:

cycle	0	1	2	3	4	5	6	7	8	9	10	11	12
Multiplication													idle
	x_2y_1	x_1y_2	$(x_1 + y_1)(x_2 + y_2)$	$x_2y_1x_1y_2$	z_1z_2	$dx_2y_1x_1y_2$	$z_1z_2z_1z_2$	$z_1z_2(x_1y_2 + x_2y_1)$	$z_1z_2(x_1x_2 + y_1y_2)$	X_3	Y_3	Z_3	
mm_x	x_2	x_1	$reg1$	mac	z_1	mac	mac	z_1	z_1	mac	mac	$reg1$	
mm_y	y_1	y_2	$sum1$	$reg0$	z_2	d	mac	$reg0$	$reg1$	$sum1$	$sum1$	$reg0$	
$save$	$reg0$	$reg1$	$reg1$	$-$	$reg2$	$reg0$	z_1	$-$	$-$	x_1	y_1	z_1	
Addition & Subtraction													
		$x_1 + y_1$	$x_2 + y_2$		$x_1y_2 + x_2y_1$	$x_1x_2 + y_1y_2$				$z_1z_2z_1z_2$- $dx_2y_1x_1y_2$	$z_1z_2z_1z_2$+ $dx_2y_1x_1y_2$		
add_x		x_1	x_2		$reg1$	$reg1$				z_1	z_1		
add_y		y_1	y_2		$reg0$	$reg0$				$reg0$	$reg0$		
addsub		0	0		0	1				1	0		
save		reg1	-		reg0	reg1				$reg1$	$reg0$		
Registers													
x_1	x_1												x_3
y_1	y_1												
z_1	z_0	z_1						z_1z_2		$z_1z_2z_1z_2$			
reg0				x_2y_1		$x_1y_2 + x_2y_1$			$dx_2y_1x_1y_2$			$z_1z_2z_1z_2$+ $dx_2y_1x_1y_2$	
reg1			$x_1 + y_1$		x_1y_2	$(x_1 + y_1)(x_2 + y_2)$	$x_1x_2 + y_1y_2$				$z_1z_2z_1z_2$-$dx_2y_1x_1y_2$		
mac			x_2y_1	x_1y_2	$(x_1 + y_1)(x_2 + y_2)$	$x_2y_1x_1y_2$	z_1z_2	$dx_2y_1x_1y_2$	$z_1z_2z_1z_2$	$z_1z_2(x_1y_2 + x_2y_1)$	$z_1z_2(x_1x_2 + y_1y_2)$	X_3	

上圖是描述$(X_1, Y_1, Z_1) + (X_2, Y_2, Z_2)$ 所需的 12 個 cycle 期間，對於乘法器和加法器的控制。Multiplication 和 Addition & Subtraction 欄目第一列都是目標，中間幾列則是輸入，最後一列是算完要儲存的 register。但兩者不同的地方在於，Multiplication 第一個 cycle 是用來選擇要計算的 data，後兩個 cycles 才去計算。Addition & Subtraction 則是第一個 cycle 就計算完畢。Registers 欄目代表每個 cycle 訊號儲存的值。

整個流程需要兩個額外的 register reg0 和 reg1，用來存暫存資料。並且可以看到有幾個 cycle 輸入是 mac 或 sim1，代表是取用剛算完的值。值得注意的是，X_3, Y_3, Z_3 分別是在 idle 和下次運算的前兩個周期才儲存進X_1, Y_1, Z_1，目的是為了節省 cycle 數，但並不會影響到下次運算的正確性，因為可以先從 mac 和 sum1 拿資料。

4. Performance Evaluation

Index	Area (mm^2)	T_{cycle}(ns)	N_{cycle}	AT(gate)	$Cycle_{MM}$	Optimization
TA	-	-	-	-	33600	-
001	186W	15	20188	56.3W	20400	Do $MM(R^8)$ only at the end of addition or doubling
002	190W	10	15668	29.8W	15600	Remove most of $MM(R^2)$
003	178W	10	7965	14.2W	7600	3-Pipeline for MM operation
004	173W	10	5965	10.3W	5600	2-Pipeline for MM operation

上面的面積和 AT 是用 02 synthesis 合出的面積計算，但還是可以用來比較不同算法的優劣，實際 APR 完面積大約會再乘上 1.2~1.5 倍。上表可以看到我們的面積沒有太大變化，是因為我們認為只用一個 256 bits 的乘法器對 AT 比較好，主要是在資料怎麼流動上優化。以下是助教的算法和我們優化過程中比較重要的 4 個階段的比較。

TA:

$Cycle_{MM}\ (pred) = (400 * 13 * 2\ +\ 400 * 2)\ *\ 3\ =\ 33600$

Version 1:

$MM(R^2)$ 改成 $MM(R^8)$ 並到 addition 後面才修正回去。

$Cycle_{MM}\ (pred) = (400 * 15\ +\ 400 * 2)\ *\ 3\ =\ 20400$

Version 2:

z^{-1} 做完先存起來。一開始先乘上 $MM(R^2)$，中間都不用計算$MM(R^2)$，到最後再把值修正。

$Cycle_{MM}\ (pred) = (400 * 12\ +\ 400)\ *\ 3\ =\ 15600$

Version 3:

MM 改成乘法加上兩次 MAC 運算，兩次 MAC 分不同 cycle 做。

$Cycle_{MM}\ (pred) = 400 * 16\ +\ 400 *\ 3\ =\ 7600$

Version 4:

MM 改成乘法加上兩次 MAC 運算，兩次 MAC 在同一個 cycle 做。

$Cycle_{MM}\ (pred) = 400 * 12\ +\ 400 *\ 2\ =\ 5600$